Project Plan

Credit Card Approval Prediction System | Project Planning Phase

---

Objective

Plan the timeline, milestones, and tasks to deliver the Credit Card Approval Prediction System from ideation to a live deployed web application.

Project Timeline

Phase	Key Activities	Duration	Deliverable
1. Brainstorming	Problem framing, empathy map, idea prioritization	Week 1	3 PDFs
2. Requirements	Functional/non-functional requirements, data requirements	Week 1	Requirement Gathering PDF
3. Design	Architecture, pipeline design, UI design wireframes	Week 2	System Design Diagram PDF
4. Planning	Timeline, milestones, risk planning	Week 2	Project Plan PDF
5. Development	Data merge + target derivation, model training, Flask app	Weeks 3–4	app.py, train_model.py, PKL models, templates
6. Testing	Unit, boundary, error-handling, pipeline integration tests	Week 5	Test Cases & Report PDF
7. Documentation	Consolidated final project report	Week 5	Project Documentation PDF
8. Demonstration	Live demo on Render, walkthrough recording	Week 6	Demo PDF + live URL

Milestones

- M1 — Requirements sign-off (end of Week 1).
- M2 — Architecture and pipeline design approved (end of Week 2).
- M3 — Target variable derived correctly; all 4 models trained and compared (mid Week 4).
- M4 — Flask app functional end-to-end; friendly/raw input toggle working (end of Week 4).
- M5 — All test cases passed (end of Week 5).
- M6 — App live on Render; public URL shared with mentor (end of Week 6).

Risks & Mitigation

Risk	Mitigation
134,203 missing OCCUPATION_TYPE values (30% of records)	Fill with 'Unknown' in train_model.py; pipeline uses handle_unknown='ignore'.
Class imbalance (most applicants approved)	Use stratify=y in train_test_split; evaluate using F1, Precision, Recall, ROC-AUC not just
Pipeline preprocessing mismatch at inference	Save full Pipeline (not bare model) so Flask feeds raw form strings directly.
Render cold-start delay (free tier)	Document expected 30–50 s warm-up in demonstration guide.
Large dataset (1M+ credit records) slow to load	Preprocessing runs only in train_model.py (offline); inference uses saved pkl only.